# OpenCV

# 컴퓨터 비전과 OpenCV

# 목표

- 디지털 이미지 및 컴퓨터 비전의 기본 원리를 깊이 이해하고, OpenCV를 활용한 이미지 처리의 기초 지식을 탄탄하게 확립합니다.
- OpenCV의 핵심 API(Application Programming Interface) 함수들을 사용하여 이미지 읽기/쓰기부터 필터링, 변환까지 효율적으로 처리하는 능력을 배양합니다.
- 경계선 검출, 윤곽선 분석, 히스토그램 평활화 등 주요 영상 처리 기법의 원리를 파악하고, 주어진 문제에 적합한 알고리즘을 선택할 수 있는 판단력을 기릅니다.
- 실시간 비디오 스트림 처리, 움직임 감지, 기본적인 객체 검출과 같은 실용적인 컴퓨터 비전 응용 프로젝트를 직접 구현하여 문제 해결 능력을 강화합니다.

# 컴퓨터 비전 (Computer Vision) 개요

- 컴퓨터 비전은 말 그대로 **컴퓨터가 볼 수 있게 하는 기술**입니다.
- **정의:** 이미지나 비디오와 같은 디지털 미디어로부터 **의미 있는 정보**를 추출하고, 그 내용을 **이해하고 해석**하는 과학 및 기술 분야입니다.
- **목표:** 인간의 시각 시스템이 세상을 인식하는 것처럼, 기계가 시각 데이터를 인식, 처리, 분석하여 자율적인 의사 결정을 내릴 수 있도록 하는 것입니다.
- **주요 작업:**
  - **이미지 분류 (Image Classification):** 이미지에 무엇이 있는지 (예: "고양이" 또는 "개") 식별합니다.
  - **객체 검출 (Object Detection):** 이미지 내에서 특정 객체의 위치를 사각형으로 표시합니다 (예: "사람은 이 영역에 있어").
  - **객체 추적 (Object Tracking):** 비디오에서 움직이는 객체의 경로를 따라갑니다.
  - **이미지 분할 (Image Segmentation):** 픽셀 단위로 객체의 경계를 정확히 구분합니다.

# OpenCV (Open Source Computer Vision Library)

- **OpenCV**는 컴퓨터 비전 분야에서 가장 널리 사용되는 **오픈 소스 라이브러리**입니다.
- **정의**: 실시간 컴퓨터 비전 애플리케이션 개발을 위해 설계된 프로그래밍 라이브러리로, C++로 작성되었지만 Python, Java 등 다양한 언어를 지원. 특히 Python과의 연동성이 뛰어나 교육 및 연구 분야에서 주로 사용.
- **역할**: 복잡한 컴퓨터 비전 알고리즘을 사용자가 쉽게 호출하여 쓸 수 있도록 표준화된 함수와 도구를 제공
- **주요 기능 및 활용 분야**:
  - 기본 이미지/비디오 처리: 이미지 읽기/쓰기/표시, 크기 조절, 회전, 색상 공간 변환 (BGR ⇔ Gray ⇔ HSV).
  - 필터링: 이미지 노이즈 제거 (블러링), 선명화, 경계선 검출 (Canny, Sobel).
  - 특징점 검출 및 매칭: 이미지 내의 고유한 지점 (코너, 엣지)을 찾아 객체 인식이나 이미지 정렬에 활용합니다.
  - 객체 검출: Haar Cascade를 이용한 얼굴 검출이나 딥러닝 기반 모델 (DNN 모듈)을 활용한 객체 인식 및 추론을 지원합니다.
  - 비디오 분석: 움직임 감지, 옵티컬 플로우 (Optical Flow) 등 동영상 스트림 분석 기능.

# OpenCV와 CNN(합성곱 신경망)의 관계

- 딥러닝을 이용한 이미지 인식 프로젝트에서 두 라이브러리는 서로 경쟁하는 것이 아니라, 각자의 역할 분담을 통해 하나의 시스템을 완성합니다.
- 역할 분담: 전처리와 핵심 연산

| 구분 | OpenCV (조수) | CNN/PyTorch (두뇌) |
|---|---|---|
| **주요 역할** | 이미지 읽기, 크기 조절, 색상 변환, 노이즈 제거 | 이미지 속 패턴 학습, 사물 분류, 특징 추출 |
| **작업 시점** | 인공지능이 이미지를 보기 **전** (전처리) | 인공지능이 이미지를 해석할 **때** (추론) |
| **데이터 형태** | NumPy 배열 (H, W, C) | 텐서 (Tensor: C, H, W) |

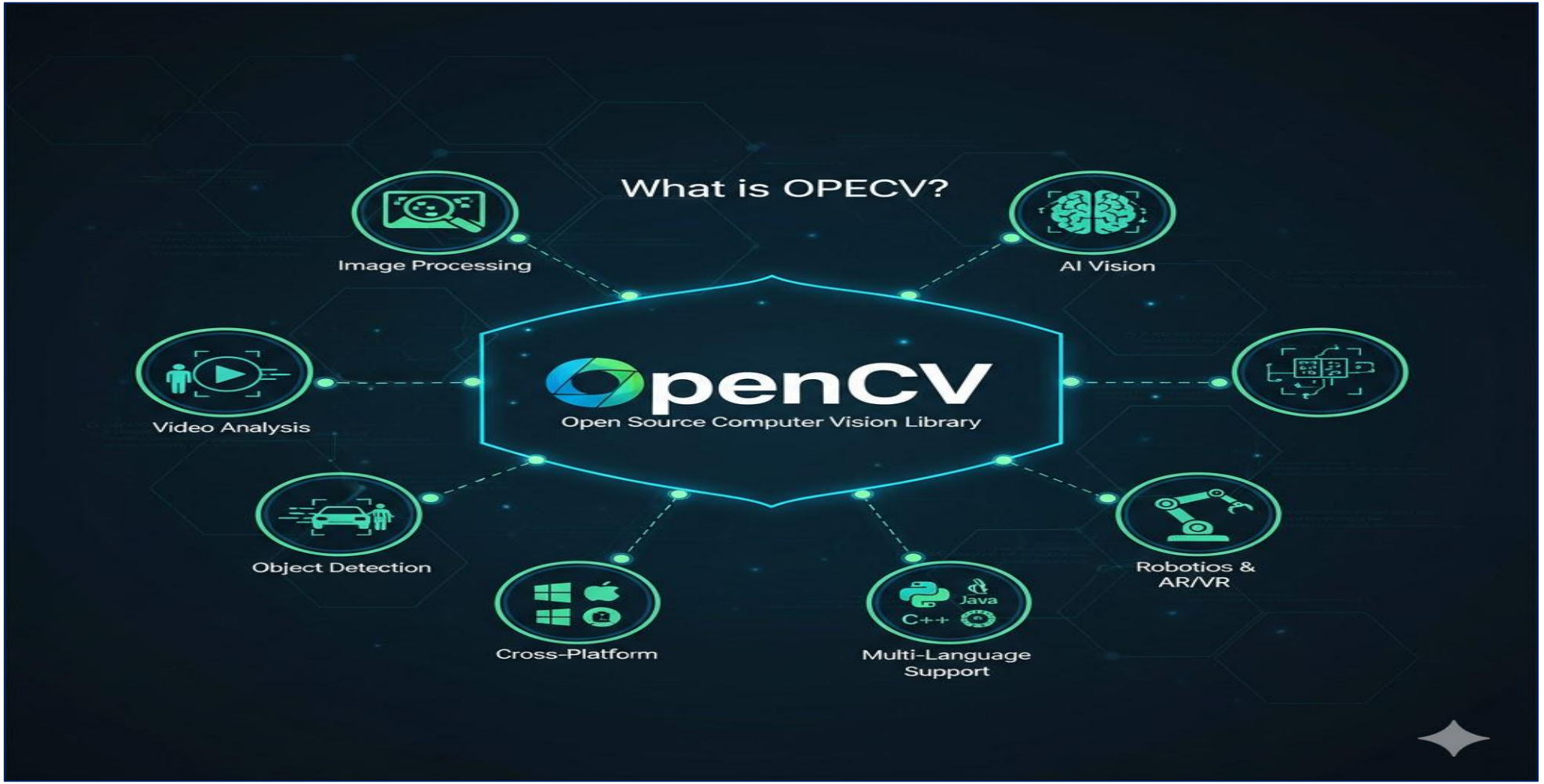
# 왜 함께 사용하나요?

- 데이터의 규격화 (Preprocessing)
  - CNN 모델은 보통 고정된 크기의 입력값을 요구합니다. (예: 224 X 224 픽셀)사용자가 넣는 이미지는 크기가 제각각입니다.OpenCV가 이 이미지들을 모델이 원하는 크기로 자르고(Crop), 늘리고(Resize), 색상을 맞추는 작업을 수행합니다.
- 노이즈 제거 및 특징
  - 강조이미지가 너무 어둡거나 노이즈가 많으면 CNN의 정확도가 떨어집니다.OpenCV를 이용해 밝기를 조절하거나 선명도를 높여 CNN이 더 쉽게 배울 수 있도록 돕습니다.
- 결과 시각화 (Post-processing)
  - CNN이 "이 사진은 강아지입니다"라고 판단하면, 그 결과를 화면에 보여줘야 합니다.OpenCV를 사용하여 사진 위에 강아지 위치를 상자(Bounding Box)로 그리거나 글자를 써서 사용자에게 보여줍니다.

# OpenCV와 dnn모듈

- **기술적 연결 고리: OpenCV의 dnn 모듈**
  - 최근 OpenCV는 단순히 전처리만 하는 것을 넘어, 이미 학습된 CNN 모델(TensorFlow, PyTorch 모델 등)을 불러와서 실행할 수 있는 cv2.dnn 모듈을 포함하고 있습니다. 덕분에 무거운 딥러닝 프레임워크 없이도 OpenCV만으로도 간단한 인공지능 기능을 구현할 수 있습니다.

# OpenCV


What is OPECV?
Image Processing
AI Vision
Video Analysis
OpenCV
Open Source Computer Vision Library
Object Detection
Robotios &
AR/VR
Cross-Platform
Java
C++
Multi-Language
Support

# OpenCV의 장점

- 오픈 소스 (Open Source): 누구나 무료로 사용할 수 있고, 방대한 커뮤니티의 지원을 받습니다.
- 고성능 (High Performance): 핵심 기능은 C/C++로 구현되어 있어 속도가 빠르며, 다중 코어 프로세싱을 활용할 수 있습니다.
- 광범위한 지원: 다양한 운영체제(Windows, Linux, macOS, Android, iOS) 및 프로그래밍 언어를 지원합니다.

# 개발 환경 설정

- **Anaconda 또는 Miniconda 설치:**
  - **Anaconda:** 데이터 과학에 필요한 대부분의 라이브러리(NumPy, Pandas 등)가 포함된 종합 배포판입니다. 초보자에게 편리합니다.
  - **Miniconda:** Python과 conda (환경 관리자)만 포함된 경량 버전입니다. 필요한 패키지만 설치하여 환경을 깨끗하게 유지할 수 있습니다.

# PowerShell에서 Conda 환경 설정하기

- Anaconda PowerShell prompt를 관리자 권한으로 열고 다음 명령어를 실행
- (Anaconda Power Shell 생김)
- conda init powershell
- 이 명령어를 실행한 후 **PowerShell 창을 닫았다가 다시 열어야** 변경 사항이 적용됩니다.
- 가상환경 생성및 활성화 예시
  - Anaconda prompt 를 관리자 권한으로 작동시킨다
  - conda create --name cv_env python=3.10
  - conda activate cv_env

# OpenCV 라이브러리 설치

- OpenCV 설치 (파이썬 3.13까지는 지원을 안하고 있음)
  - conda create -n vision_env python=3.10 –y
  - conda activate vision_env
  - conda install pytorch torchvision torchaudio cpuonly opencv -c pytorch -c matplotlib conda-forge

# 개발 환경 설정

- VS Code 또는 PyCharm 설치/실행
- **인터프리터 설정:** 에디터 내에서 방금 생성하고 활성화한 **cv_env 가상 환경의 Python 인터프리터**를 사용하도록 설정합니다.
- ctrl-shift-p Python:select Interpreter

# OpenCV 기본 구조 이해

- 파이썬 환경에서 OpenCV는 이미지를 **NumPy 배열** 형태로 처리합니다.
- 픽셀(Pixel)과 이미지 데이터
  - 컴퓨터 비전에서 이미지는 픽셀(Pixel)이라는 가장 작은 점들로 이루어진 격자(Grid)입니다.
  - 픽셀은 이미지를 구성하는 최소 단위이며, 각 픽셀은 밝기와 색상 정보를 숫자로 가지고 있습니다.
  - 해상도는 이미지의 가로 픽셀 수와 세로 픽셀 수 (예: 1920 X 1080). 해상도가 높을수록 더 많은 픽셀을 포함하며 이미지가 상세해집니다.

# OpenCV의 데이터 구조

- OpenCV가 파이썬에서 이미지를 불러오거나 처리하면, 그 결과는 항상 NumPy의 N-차원 배열 (ndarray) 형태가 됩니다.

- 흑백 이미지 (Gray Scale)
  - 흑백(그레이 스케일) 이미지는 밝기 정보만 가집니다.
  - 차원 : 2차원 배열 (행 X 열).
  - 픽셀 값: 0 (검은색)부터 255 (흰색)까지의 단일 숫자 값.
  - 예시: 512 X 512 해상도의 흑백 이미지 => (512, 512) 크기의 2차원 배열.

# OpenCV의 데이터 구조

- 컬러 이미지 (Color Image)
  - 컬러 이미지는 색상을 표현하기 위해 여러 개의 채널(Channel)을 사용합니다.
  - 차원: 3차원 배열 (행 X 열 X 채널 수).
  - 픽셀 값: 각 픽셀은 3개의 숫자 값으로 색상을 표현합니다.
  - OpenCV의 채널 순서 (BGR):OpenCV는 대부분의 경우 이미지를 **Blue (파랑), Green (초록), Red (빨강)** 순서의 3개 채널로 저장합니다.
  - (일반적인 웹/그래픽 환경의 RGB 순서와 다름에 주의해야 합니다.)
  - 예시: 512 X 512 해상도의 컬러 이미지 => (512, 512, 3) 크기의 3차원 배열.

# 기본 이미지 조작 함수

| 기능 | OpenCV 함수 | 설명 |
|---|---|---|
| **읽기** | cv2.imread(filepath, flag) | 이미지 파일을 불러와 **NumPy 배열**로 반환합니다. flag로 흑백/컬러 등을 지정할 수 있습니다. |
| **표시** | cv2.imshow(winname, mat) | 이미지를 팝업 창에 표시합니다. |
| **저장** | cv2.imwrite(filepath, mat) | NumPy 배열 형태의 이미지를 파일로 저장합니다. |
| **대기** | cv2.waitKey(delay) | 키 입력을 대기합니다. delay가 0이면 무한 대기합니다. |
| **창 닫기** | cv2.destroyAllWindows() | 생성된 모든 이미지 창을 닫습니다. |

# 첫번째 예제(기본.py)

```python
import cv2
import numpy as np
import os # 파일 존재 확인용

# 1. 사용할 이미지 파일 경로 설정
file_path = './images/sample.jpg'

# 2. 이미지 읽기 (컬러: 기본값)
# 이미지 파일이 NumPy 배열로 로드됩니다.
img_color = cv2.imread(file_path)
print(f"원본 이미지 shape: {img_color.shape}")

# 3. 이미지 표시
# 'Color Image'라는 이름의 창에 원본 이미지를 표시합니다.
cv2.imshow('Color Image', img_color)

# 4. 키 입력 대기
# '0'은 키 입력이 있을 때까지 무한정 대기하라는 의미입니다.
cv2.waitKey(0)

# 5. 모든 창 닫기
cv2.destroyAllWindows()
```

# 기본2.py

- 개선

```
# 파일이 존재하는지 확인 (없으면 에러 방지)
if not os.path.exists(file_path):
    print(f"오류: '{file_path}' 파일을 찾을 수 없습니다. 경로를 확인해주세요.")
    # 임의의 검은색 이미지 생성 (파일이 없을 경우)
    img_color = np.zeros((300, 400, 3), dtype=np.uint8)
    cv2.putText(img_color, "No Image Found", (50, 150),
                cv2.FONT_HERSHEY_SIMPLEX, 1,
                (255, 255, 255), 2)
    file_name_out = 'output_no_file.jpg'
else:
    # 2. 이미지 읽기 (컬러: 기본값)
    # 이미지 파일이 NumPy 배열로 로드됩니다.
    img_color = cv2.imread(file_path)
    print(f"원본 이미지 shape: {img_color.shape}")
    file_name_out = 'output_gray.jpg'
```

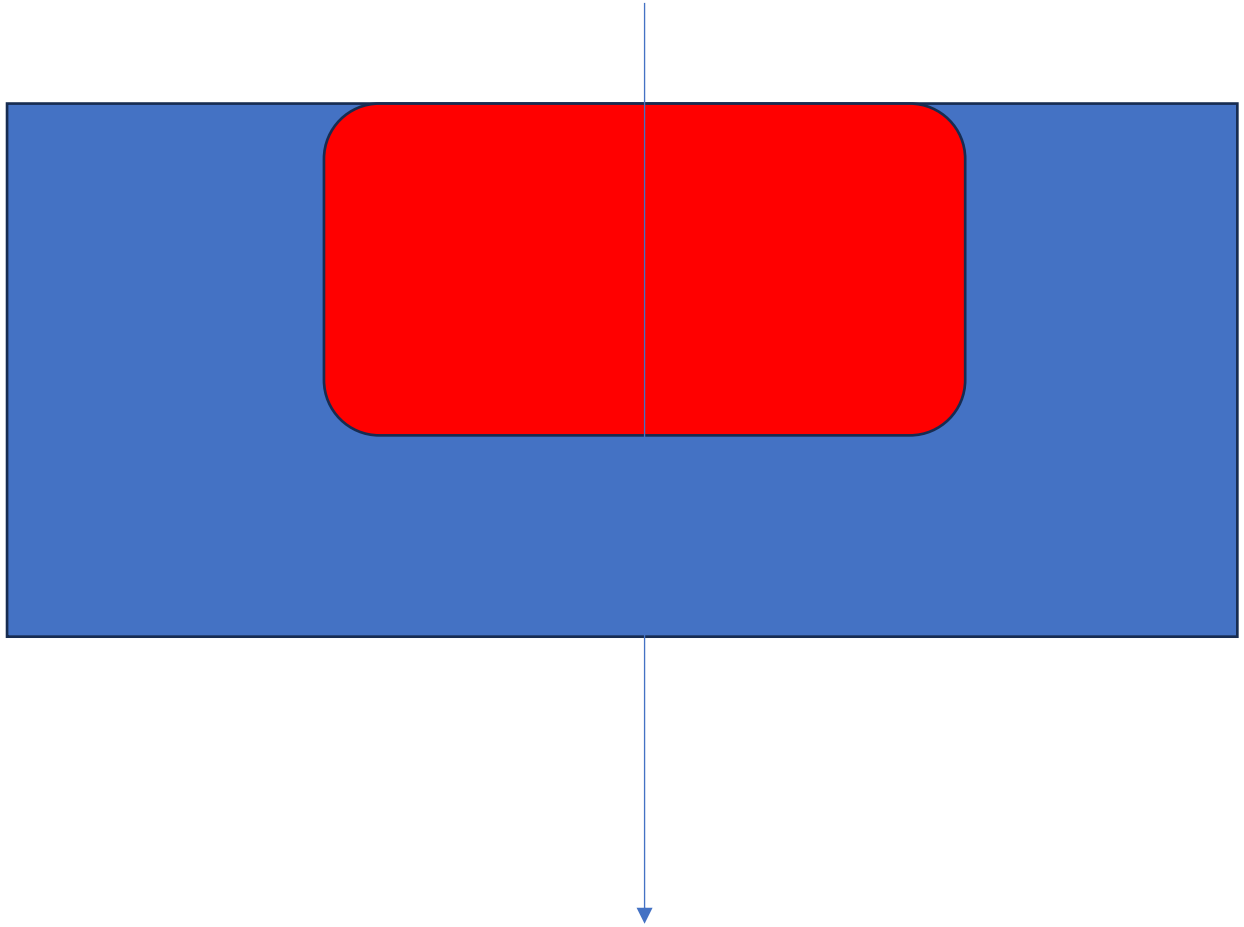
# 기본3.py

- 컬러아미지 => 흑백이미지로 변환

```
# 4. 색상 공간 변환 (BGR -> Gray Scale)
# 컬러 이미지를 흑백 이미지로 변환합니다.
img_gray = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)
print(f"흑백 이미지 shape: {img_gray.shape}") # 3차원에서 2차원으로 변경됨
```

# 양쪽에 여백구하기

- (배경크기 – 출력할이미지크기) /2



# 픽셀 접근 및 기본 조작

- OpenCV에서 이미지를 다룬다는 것은 곧 **NumPy 배열**을 다룬다는 의미입니다.
- 픽셀 접근 및 조작은 이미지의 특정 위치나 영역을 변경하기위해 사용합니다.
- **픽셀 값 접근 및 수정**
  - 이미지 배열은 행(높이), 열(너비), 채널 순서의 3차원 구조를 갖습니다.
  - **컬러 이미지 (BGR):** img[y, x, c]
  - **흑백 이미지 (Gray Scale):** img[y, x]
  - **단일 픽셀 값 읽기 (Read)**
    - 특정 좌표 (x=200, y=100)의 **파란색(B)** 채널 값을 읽는 예시입니다.

```
# img가 컬러 이미지(BGR)일 경우
y, x = 100, 200
blue_value = img[y, x, 0] # 0은 B (Blue) 채널 인덱스
print(f"B 값: {blue_value}")
```

# 픽셀 접근 및 기본 조작

- **단일 픽셀 값 수정 (Write)**
- 동일한 좌표의 **모든 채널(BGR)** 값을 **순수한 빨간색**으로 설정하는 예시입니다. (OpenCV는 BGR 순서: [B, G, R])

```
y, x = 100, 200
img[y, x] = [0, 0, 255] # B=0, G=0, R=255
```

- NumPy 배열 연산은 매우 빠르지만, 반복문(for 루프)을 사용하여 픽셀 하나하나를 순회하며 접근하고 수정하는 것은 속도가 매우 느립니다. 대부분의 이미지 처리는 **NumPy 슬라이싱**이나 **OpenCV/NumPy 함수**를 사용하는 것이 권장됩니다.

# 픽셀 접근 및 기본 조작

- 관심 영역 (ROI: Region of Interest) 설정
  - ROI는 이미지의 특정 직사각형 영역을 추출하거나 조작할 때 사용됩니다. 이는 **NumPy의 배열 슬라이싱** 기능을 그대로 활용합니다.

$$\text{ROI} = \text{img}[\text{시작_행 (y1)} : \text{끝_행 (y2)}, \text{시작_열 (x1)} : \text{끝_열 (x2)}]$$

  - **ROI 추출 및 복사** : 이미지의 일부를 잘라내어 새로운 변수(roi)에 저장합니다.

```
# (100, 50) 부터 (300, 250) 영역을 추출
roi = img[100:300, 50:250]
```

# 픽셀 접근 및 기본 조작

- **ROI를 이용한 붙여넣기 :** 추출된 ROI를 원본 이미지의 다른 위치에 복사합니다. 이 방법은 이미지의 크기가 동일해야 합니다.

```
# 추출된 roi를 (400, 300) 위치에 덮어쓰기
img[400:600, 300:500] = roi
```

- **ROI를 이용한 색상 변경 :** ROI를 설정한 후 해당 영역의 모든 픽셀 값을 한 번에 변경할 수 있습니다.

```
# 이미지 상단 100x100 영역을 녹색으로 변경
img[0:100, 0:100] = [0, 255, 0] # B=0, G=255, R=0 (녹색)
```

# 픽셀 접근 및 기본 조작






# 픽셀 접근 및 기본 조작 함수

| 함수 | 용도 | 설명 |
| --- | --- | --- |
| **cv2.cvtColor()** | 색상 공간 변환 | BGR ->Gray, BGR->HSV 등 이미지의 색상 표현 방식을 변경합니다. |
| **cv2.split()** | 채널 분리 | 컬러 이미지를 **B, G, R** 등 개별 **2차원 배열(채널)**로 분리합니다. |
| **cv2.merge()** | 채널 재결합 | 분리된 2차원 채널들을 다시 3차원 컬러 이미지로 합칩니다. |
| **cv2.inRange()** | 색상 마스크 생성 | 특정 색상 범위(주로 HSV 공간)에 해당하는 픽셀만 추출하는 **이진 마스크**를 만듭니다. |

# 산술및 비트연산

| 함수 | 용도 | 설명 |
| --- | --- | --- |
| **cv2.add()** | 덧셈 연산 | 두 이미지 또는 이미지와 상수를 더해 **밝기를 증가**시킵니다. 픽셀 값이 255를 초과하지 않도록 자동 처리합니다. |
| **cv2.subtract()** | 뺄셈 연산 | 두 이미지 또는 이미지와 상수를 빼서 **밝기를 감소**시킵니다. 픽셀 값이 0 미만이 되지 않도록 자동 처리합니다. |
| **cv2.bitwise_and()** | 논리곱 연산 (마스킹) | 원본 이미지와 마스크를 논리곱(AND)하여, **마스크 영역만** 원본 이미지를 유지하도록 필터링합니다. |
| **NumPy 연산자** | 빠른 산술 연산 | img + 50, img * 2와 같이 NumPy의 **벡터화된 연산자**를 사용하여 픽셀 값을 직접 조작합니다. (단, 0 ~255 범위를 초과할 경우 경고 없이 오버플로우될 수 있어 cv2 함수가 더 안전할 때도 있습니다.) |

# 기본 이미지 처리 및 변환

# 도형 그리기 및 텍스트 출력

- OpenCV는 이미지 위에 **직접 도형을 그리거나 텍스트를 출력**하는 기능을 제공합니다. 이는 디버깅, 객체 검출 결과 시각화(바운딩 박스), 또는 사용자 인터페이스를 만드는 데 필수적입니다. 모든 그리기 작업은 원본 이미지의 **NumPy 배열을 직접 수정**합니다.

# 기본도형그리기및 텍스트 출력 함수

| 함수 | 도형 | 매개변수 |
|---|---|---|
| **cv2.line()** | 선 | **(시작 좌표, 끝 좌표, 색상, 두께)** |
| **cv2.rectangle()** | 사각형 | **(좌상단 좌표, 우하단 좌표, 색상, 두께)** |
| **cv2.circle()** | 원 | **(중심 좌표, 반지름, 색상, 두께)** |
| **cv2.ellipse()** | 타원 | **(중심 좌표, 장축/단축 길이, 회전 각도, 시작 각도, 끝 각도, 색상, 두께)** |
| **cv2.polylines()** | 다각형 (선) | **(꼭짓점 배열, 닫힘 여부, 색상, 두께)** |
| **cv2.fillPoly()** | 다각형 (채움) | **(꼭짓점 배열, 색상)** |

| 함수 | 용도 | 매개변수 |
|---|---|---|
| **cv2.putText()** | 텍스트 출력 | **(이미지, 텍스트, 시작 좌표, 폰트 종류, 폰트 크기, 색상, 두께)** |

# 폰트종류

- cv2.FONT_HERSHEY_SIMPLEX
- cv2.FONT_HERSHEY_DUPLEX
- cv2.FONT_HERSHEY_COMPLEX
- cv2.FONT_ITALIC (기존 폰트에 이탤릭체 추가)

# 도형1.py

```python
import cv2
import numpy as np

# 1. 500x500 크기의 검은색 배경 이미지 생성
# 3채널 (컬러), 픽셀 값 타입은 8비트 정수 (0~255)
img = np.zeros((500, 500, 3), dtype=np.uint8)

# 2. 선 그리기: 대각선 (파란색)
cv2.line(img, (0, 0), (500, 500), (255, 0, 0), 2) # B=255, G=0, R=0, 두께 2

# 3. 사각형 그리기: 중심부 (녹색, 채우기)
# (100, 100)부터 (400, 400)까지
cv2.rectangle(img, (100, 100), (400, 400), (0, 255, 0), -1)

# 4. 원 그리기: 중심부 (빨간색)
# 중심 (250, 250), 반지름 50
cv2.circle(img, (250, 250), 50, (0, 0, 255), 5)

# 5. 텍스트 출력: 제목 (흰색)
text = "OpenCV Drawing Test"
# 시작 좌표 (50, 50), 폰트 크기 1, 두께 2
cv2.putText(img, text, (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

# 6. 이미지 표시
cv2.imshow('Drawing Canvas', img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

# OpenCV 마우스 이벤트 처리

- OpenCV에서 마우스 이벤트를 처리하는 데 사용되는 핵심 함수는 **콜백 함수를 등록**하는 **cv2.setMouseCallback()** 함수입니다. 이 함수는 이미지 창에서 발생하는 다양한 마우스 동작(이벤트)을 감지하고, 해당 이벤트 발생 시 미리 정의된 사용자 함수를 호출하도록 연결해 줍니다.

| 함수 | 용도 | 구문 |
|---|---|---|
| **cv2.namedWindow()** | 창 이름 설정 | `cv2.namedWindow('창 이름')` |
| **cv2.setMouseCallback()** | 이벤트 콜백 등록 | `cv2.setMouseCallback('창 이름', 콜백_함수)` |

# 콜백함수의구조

- cv2.setMouseCallback()에 등록하는 콜백 함수는 항상 다음 네 가지 인자를 받도록 정의해야 합니다.

```
def callback_function(event, x, y, flags, param):
    # event: 발생한 마우스 이벤트의 종류 (아래 표 참고)
    # x, y: 마우스 포인터의 현재 좌표
    # flags: 마우스 버튼의 현재 상태 (예: Ctrl 키, Shift 키 등)
    # param: setMouseCallback 호출 시 전달된 추가 데이터 (거의 사용 안 함)
    pass
```

# 주요 이벤트 상수

| 상수 | 설명 |
|---|---|
| **cv2.EVENT_LBUTTONDOWN** | 마우스 **왼쪽 버튼**을 누르는 순간 |
| **cv2.EVENT_RBUTTONDOWN** | 마우스 **오른쪽 버튼**을 누르는 순간 |
| **cv2.EVENT_MBUTTONDOWN** | 마우스 **가운데 버튼**을 누르는 순간 |
| **cv2.EVENT_LBUTTONUP** | 마우스 **왼쪽 버튼**을 놓는 순간 |
| **cv2.EVENT_MOUSEMOVE** | 마우스를 **움직이는** 동안 |
| **cv2.EVENT_LBUTTONDBLCLK** | 마우스 **왼쪽 버튼**을 **더블 클릭**했을 때 |
| **cv2.EVENT_RBUTTONDBLCLK** | 마우스 **오른쪽 버튼**을 **더블 클릭**했을 때 |
| **cv2.EVENT_MOUSEWHEEL** | 마우스 **휠을 돌릴 때** (Windows와 macOS에서는 잘 작동하지 않을 수 있음) |

# 마우스 플래그 상수 (Flags)

| 상수 | 설명 |
|---|---|
| **cv2.EVENT_FLAG_CTRLKEY** | Ctrl 키가 눌린 상태 |
| **cv2.EVENT_FLAG_SHIFTKEY** | Shift 키가 눌린 상태 |
| **cv2.EVENT_FLAG_ALTKEY** | Alt 키가 눌린 상태 |
| **cv2.EVENT_FLAG_LBUTTON** | 왼쪽 버튼이 눌린 상태 (드래그 감지) |
| **cv2.EVENT_FLAG_RBUTTON** | 오른쪽 버튼이 눌린 상태 |
| **cv2.EVENT_FLAG_MBUTTON** | 가운데 버튼이 눌린 상태 |

# 이벤트처리, 이벤트1.py

```python
import cv2
import numpy as np

# 1. 마우스 이벤트 콜백 함수 정의
def on_mouse(event, x, y, flags, param):
    # 마우스 왼쪽 버튼을 누를 때 (LBUTTONDOWN)
    if event == cv2.EVENT_LBUTTONDOWN:
        # 클릭된 좌표(x, y) 출력
        print(f"마우스 클릭 좌표: x={x}, y={y}")

        # 클릭된 위치에 작은 파란색 원 그리기
        # (이미지, 중심 좌표(x, y), 반지름, 색상(BGR), 두께(-1은 채우기))
        cv2.circle(img, (x, y), 5, (255, 255, 255), -1)

        # 변경된 이미지를 즉시 창에 반영
        cv2.imshow('Image', img)

# 2. 검은색 배경 이미지 생성
# 500x500 크기의 3채널(컬러) 이미지
img = np.zeros((500, 500, 3), dtype=np.uint8)

# 3. 창 설정 및 콜백 함수 등록
cv2.namedWindow('Image')
# 'Image' 창에 on_mouse 함수를 이벤트 핸들러로 등록
cv2.setMouseCallback('Image', on_mouse)

# 4. 이미지 표시 및 대기
cv2.imshow('Image', img)
cv2.waitKey(0) # 키 입력 대기 (창이 닫히지 않도록 함)
cv2.destroyAllWindows()
```

# 색상 공간 변환 및 임계 처리

- **색상 공간 변환 (Color Space Conversion)**
- 색상 공간은 색상을 표현하는 방식(모델)을 의미합니다.
- 1. BGR -> Gray Scale (흑백)
  - 용도: 이미지의 밝기 정보만 추출하여 연산량을 줄이고, 엣지 검출이나 특징점 검출 등 밝기에만 의존하는 알고리즘에 사용됩니다.
  - 변환: 컬러 이미지(3채널)를 흑백 이미지(1채널)로 변환합니다.

```
# BGR -> 흑백 변환
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

# 색상 공간 변환 및 임계 처리

- BGR ↔ HSV
  - 용도: 색상(Hue)을 채도나 밝기로부터 분리하여 표현하기 때문에, 특정 색상(예: 빨간색 공)을 기반으로 객체를 검출할 때 매우 효과적입니다.
  - H (Hue): 색상 자체 (0° ~ 180° 또는 0 ~ 255)
  - S (Saturation): 채도 (색의 순도)
  - V (Value): 명도/밝기
  - 변환: BGR에서 HSV로 변환 후, cv2.inRange() 함수를 이용해 특정 색상 범위를 쉽게 분리할 수 있습니다.

```
# BGR -> HSV 변환
hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

# 색상 공간 변환 및 임계 처리

- **임계 처리 (Thresholding)**
  - 임계 처리는 흑백 이미지의 픽셀 값을 특정 기준값(Threshold)과 비교하여, 이미지 전체를 **이진 이미지(Binary Image)**, 즉 픽셀 값이 0(검은색) 또는 255(흰색)만 가지는 이미지로 만드는 과정입니다.
  - **목표:** 이미지에서 관심 영역(객체)과 배경을 명확하게 분리합니다.

# 색상 공간 변환 및 임계 처리

- **1) 단순 임계 처리 (Simple Thresholding)**
- 전체 이미지에 **하나의 고정된 기준값**을 적용합니다.

| 함수 | cv2.threshold(src, thresh, maxval, type) |
|---|---|
| src | 입력 이미지 (반드시 **Gray Scale**이어야 함) |
| thresh | 기준값 (Threshold value) |
| maxval | 기준값을 넘었을 때 할당할 최댓값 (보통 255) |
| type | 임계 처리 유형 (예: cv2.THRESH_BINARY) |

- **주요 임계 처리 유형:**
  - cv2.THRESH_BINARY: 픽셀 값이 기준값보다 크면 maxval(255), 작거나 같으면 0.
  - cv2.THRESH_BINARY_INV: 위와 반대 (크면 0, 작거나 같으면 255).

# 색상 공간 변환 및 임계 처리

- **적응형 임계 처리 (Adaptive Thresholding)**
- 이미지의 밝기가 부분적으로 크게 다를 때 유용합니다. 이미지 영역을 작은 블록으로 나누어 **각 블록마다 기준값을 다르게** 설정합니다.

| 함수 | cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C) |
|---|---|
| adaptiveMethod | 블록 내에서 기준값을 결정하는 방법 (예: 평균, 가우시안) |
| blockSize | 임계값을 계산할 이웃 픽셀의 크기 (홀수) |
| C | 계산된 평균/가중 평균에서 뺄 상수 |

# 색상 공간 변환 및 임계 처리

- **오츠의 임계 처리 (Otsu's Thresholding)**
- 사용자가 기준값을 직접 지정할 필요 없이, 이미지의 **히스토그램**을 분석하여 배경과 객체를 가장 잘 분리할 수 있는 **최적의 기준값**을 자동으로 찾아 적용합니다.

```
# Otsu's 방법을 사용하려면 type에 cv2.THRESH_OTSU를 추가하고, thresh 값은 0으로 설정
ret, binary_img = cv2.threshold(gray_img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OT
```

# 임계처리1.py

```python
import cv2
import numpy as np

file_path = './images/sample2.jpg'
img_color = cv2.imread(file_path)

if img_color is None:
    print(f"오류: '{file_path}' 파일을 로드할 수 없습니다.")
    exit()

# 1. BGR -> Gray Scale 변환
img_gray = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)

# 2. 단순 임계 처리 (Threshold = 127)
# 픽셀 값이 127보다 크면 255(흰색), 아니면 0(검은색)
ret, img_binary = cv2.threshold(img_gray, 127, 255, cv2.THRESH_BINARY)

# 3. 오츠 임계 처리 (자동 기준값)
# ret_otsu 변수에 최적의 기준값이 저장됨
ret_otsu, img_otsu = cv2.threshold(img_gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 4. 결과 이미지 표시
cv2.imshow('Original Color', img_color)
cv2.imshow('Gray Scale', img_gray)
cv2.imshow('Simple Binary (T=127)', img_binary)
cv2.imshow(f'Otsu Binary (T={ret_otsu:.0f})', img_otsu)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

# 기하학적 변환

- 기하학적 변환은 이미지의 **픽셀 위치를 변경**하여 **크기, 모양, 방향 등을 조작하는 것을** 의미합니다. 객체의 크기를 맞추거나, 여러 이미지를 정렬하거나, 왜곡된 이미지를 보정할 때 필수적으로 사용됩니다.
- 주요 함수는 **cv2.warpAffine()** (어파인 변환)과 **cv2.warpPerspective()** (원근 변환)이며, 이 함수들을 사용하려면 변환을 정의하는 변환 행렬(Transformation Matrix)을 먼저 계산해야 합니다.

# 기하학적 변환

- 크기 변환 (Scaling) : 이미지의 크기를 확대하거나 축소합니다.

| 함수 | 용도 | 구문 |
|---|---|---|
| **cv2.resize()** | 이미지 크기 변경 | cv2.resize(src, dsize, fx, fy, interpolation) |

  - dsize (Desired Size): 변경하고자 하는 출력 크기 (튜플, 예: (가로, 세로))
  - fx, fy: 가로 및 세로 방향 스케일링 비율
  - interpolation (보간법): 픽셀 값을 채우는 방법 (이미지 품질 결정)
  - cv2.INTER_LINEAR (기본값): 속도가 빠르고 품질이 좋음 (축소/확대 시)
  - cv2.INTER_CUBIC: 품질이 가장 좋음 (주로 확대 시)
  - cv2.INTER_AREA: 축소 시 가장 좋은 결과를 보임

# 기하학적 변환

- 이동 변환 (Translation)이미지를 x축과 y축 방향으로 평행하게 이동시킵니다.
- 이동 거리를 정의하는 2 X 3 변환 행렬(M)을 사용합니다.

$$M = \begin{pmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \end{pmatrix}$$

($T_x$: $x$축 이동 거리, $T_y$: $y$축 이동 거리)

| 함수 | 용도 | 구문 |
|---|---|---|
| **cv2.warpAffine()** | 어파인 변환 적용 | cv2.warpAffine(src, M, dsize) |

# 기하학적 변환

- **회전 변환 (Rotation)**
  - 이미지를 특정 중심점을 기준으로 회전시킵니다.
  - **회전 행렬 계산:** cv2.getRotationMatrix2D()
  - **변환 적용:** cv2.warpAffine()

| 함수 | 용도 | 구문 |
|---|---|---|
| **cv2.getRotationMatrix2D()** | 회전 행렬 계산 | cv2.getRotationMatrix2D(center, angle, scale) |
| **center:** 회전의 중심 좌표 (예: (w/2, h/2)) | | |
| **angle:** 회전 각도 (도 Degree, 양수는 반시계 방향) | | |
| **scale:** 스케일링 비율 (1.0이면 크기 유지) | | |

# 기하학적 변환

- **어파인 변환 (Affine Transformation)**
  - 이동, 회전, 크기, 전단(Shear) 변환이 모두 포함된 일반적인 선형 변환입니다. 직선은 그대로 직선으로 유지되지만, 평행선은 평행을 유지하지 않을 수 있습니다.핵심: 3개의 쌍이 되는 점(원본 이미지의 3점과 변환될 이미지의 3점)을 사용하여 변환 행렬($M$)을 계산합니다.

| 함수 | 용도 | 구문 |
|---|---|---|
| **cv2.getAffineTransform()** | 어파인 행렬 계산 | cv2.getAffineTransform(src_points, dst_points) |
| **cv2.warpAffine()** | 변환 적용 | cv2.warpAffine(src, M, dsize) |

# 기하학적 변환

- **원근 변환 (Perspective Transformation)**
  - 이동, 회전, 크기 외에 깊이 정보를 반영하여 원근감이 있는 변환을 수행합니다. 직선은 직선으로 유지되지만, 평행선은 소실점(Vanishing Point)으로 수렴할 수 있습니다. (예: 평면 이미지를 3차원 공간에서 본 것처럼 변환)핵심: 4개의 쌍이 되는 점을 사용하여 변환 행렬($M$)을 계산합니다.

| 함수 | 용도 | 구문 |
| --- | --- | --- |
| **cv2.getPerspectiveTransform()** | 원근 행렬 계산 | cv2.getPerspectiveTransform(src_points, dst_points) |
| **cv2.warpPerspective()** | 변환 적용 | cv2.warpPerspective(src, M, dsize) |

# 필터링 및 노이즈 제거

- 이미지 필터링은 이미지에서 원치 않는 정보(노이즈)를 줄이거나, 특정 특징(엣지, 경계)을 강조하기 위해 사용되는 핵심적인 영상 처리 기술입니다. 이는 주로 **컨볼루션(Convolution)** 연산을 통해 수행됩니다.
- **컨볼루션(Convolution)의 이해**
  - 컨볼루션은 이미지의 각 픽셀에 대해 주변 픽셀들을 함께 고려하여 새로운 픽셀 값을 결정하는 과정입니다.
  - 커널(Kernel) 또는 마스크(Mask): 컨볼루션 연산에 사용되는 작은 행렬(예: 3 X 3 또는 5 X 5)입니다. 커널의 값은 필터의 특성(블러링, 샤프닝 등)을 정의합니다.
  - 작동 방식: 커널을 이미지의 각 픽셀에 겹쳐 놓고, 커널의 값과 겹쳐진 이미지 픽셀 값을 곱한 후 모두 더하여(가중 평균) 출력 이미지의 새로운 픽셀 값으로 만듭니다.

# 필터링 및 노이즈 제거

- **노이즈 제거 필터 (블러링: Blurring)**
- 노이즈 제거는 주로 이미지의 세부 사항이나 엣지를 흐릿하게(블러링) 만들어 노이즈를 완화합니다.
  - **평균 필터 (Averaging Blur) :** 커널 영역 내 모든 픽셀 값의 **단순 평균**을 계산하여 중앙 픽셀의 값으로 대체합니다.

| 함수 | `cv2.blur(src, ksize)` |
|---|---|
| `ksize` | 커널의 크기 (예: `(5, 5)`) |

  - **가우시안 필터 (Gaussian Blur)**
    - 단순 평균과 달리, 커널 내의 픽셀에 가중치(중앙 픽셀에 더 높은 가중치)를 적용하여 평균을 계산합니다. 노이즈 제거 효과가 좋으면서도 엣지 보존 능력이 비교적 우수합니다.

# 필터링 및 노이즈 제거

| 함수 | cv2.GaussianBlur(src, ksize, sigmaX) |
|---|---|
| sigmaX | x 방향의 가우시안 표준 편차 (보통 0으로 설정하면 커널 크기로부터 자동 계산됨) |

- **미디언 필터 (Median Blur)**
  - 커널 영역 내 모든 픽셀 값 중 중앙값(Median)을 선택하여 중앙 픽셀을 대체합니다. 특히 **소금-후추 노이즈(Salt-and-Pepper Noise)** 제거에 탁월한 효과를 보입니다.

| 함수 | `cv2.medianBlur(src, ksize)` |
|---|---|
| `ksize` | 커널의 크기 (반드시 **홀수**여야 함) |

# 필터링 및 노이즈 제거

- **엣지 강조 필터 (샤프닝: Sharpening)**
  - 노이즈 제거 필터와 달리, 이미지의 엣지나 미세한 디테일을 강조하여 선명도를 높이는 필터입니다.
  - 작동 방식: 일반적으로 원본 이미지에서 블러링된 이미지를 뺀 후, 그 차이(엣지 정보)를 다시 원본 이미지에 더하여 엣지를 강조합니다. 이는 컨볼루션 연산을 통해 직접 구현될 수도 있습니다.커널 예시: 엣지를 강조하는 샤프닝 커널의 예입니다

    $$ \\ \begin{pmatrix} 0 & -1 & 0 \\ \-1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

    $$$$

  - 함수: cv2.filter2D(src, ddepth, kernel) 함수를 사용하여 사용자 정의 커널을 이미지에 적용할 수 있습니다.

# 영상 처리 기법

# 모폴로지(형태학,구조학) 연산

- 모폴로지 연산은 주로 이진 이미지(Binary Image)나 흑백 이미지(Gray Scale Image)에서 객체의 **모양이나 구조**를 단순화, 보정 또는 분석하는 데 사용됩니다.
- 이 연산은 구조 요소(Structuring Element)라는 작은 커널을 이미지에 슬라이딩하며 적용합니다.
- 가장 기본이 되는 연산은 팽창(Dilation)과 침식(Erosion)이며, 이를 조합하여 개폐(Opening/Closing) 등의 고급 연산을 수행합니다.
- **이미지 속 객체의 모양을 변형시켜** 노이즈를 제거하거나 객체의 경계를 다듬는 연산.

# 모폴로지 연산

- **구조 요소 (Structuring Element)**
- 모폴로지 연산의 결과를 결정하는 커널(마스크)입니다.

| 함수 | 용도 | 설명 |
|---|---|---|
| **cv2.getStructuringElement()** | 구조 요소 생성 | 연산에 사용할 커널의 모양과 크기를 정의합니다 |

- 모양: cv2.MORPH_RECT (사각형), cv2.MORPH_ELLIPSE (타원), cv2.MORPH_CROSS (십자형)
- 크기: (width, height) 튜플로 지정

# 모폴로지 연산

- **기본 모폴로지 연산**
  - **침식 (Erosion)**
    - **원리:** 구조 요소가 덮는 영역의 모든 픽셀이 **흰색(255)일 때만** 중앙 픽셀을 흰색으로 유지하고, **하나라도 검은색(0) 픽셀**이 있으면 중앙 픽셀을 검은색(0) 으로 만듭니다.
    - **효과:** 객체의 경계가 수축(Shrink)되어 객체 크기가 **줄어듭니다**. 객체 내부의 작은 구멍은 커지고, 작은 노이즈(흰색 점) 는 제거됩니다. 쓸데없는 점들 제거

| 함수 | cv2.erode(src, kernel, iterations) |
|---|---|
| kernel | 구조 요소 |
| iterations | 반복 횟수 (횟수가 증가할수록 침식 강도가 높아짐) |

# 모폴로지 연산

- **팽창 (Dilation)**
  - **원리:** 구조 요소가 덮는 영역의 픽셀 중 **하나라도 흰색(255) 픽셀**이 있으면 중앙 픽셀을 **흰색(255)**으로 만듭니다.
  - **효과:** 객체의 경계가 확장(Expand)되어 객체 크기가 **늘어납니다**. 객체 내부의 작은 구멍은 메워지고, 끊어진 선이 연결됩니다. 안에 구멍을 매꾼다

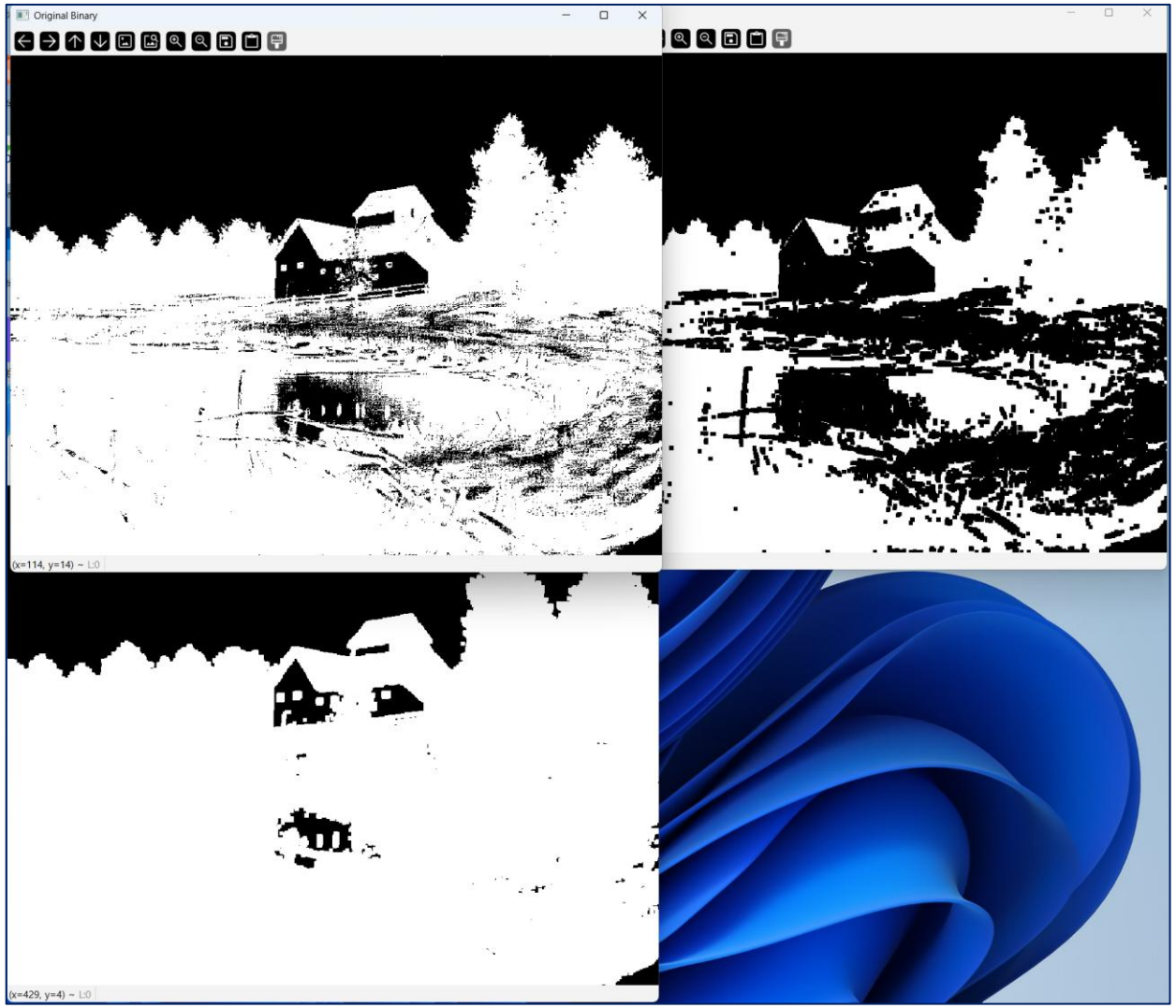
| 함수 | cv2.dilate(src, kernel, iterations) |
|---|---|
| kernel | 구조 요소 |
| iterations | 반복 횟수 |

# 모폴로지 연산

- **복합 모폴로지 연산**
  - 팽창과 침식을 조합하여 더 복잡한 효과를 얻습니다.

| 연산 | 조합 | 효과 | 함수 |
|---|---|---|---|
| **열림 (Opening)** | **침식 후 팽창** | 작은 객체(흰색 노이즈)를 제거하고 객체의 **외곽을 부드럽게** 만듭니다. | cv2.morphologyEx(src, cv2.MORPH_OPEN, kernel) |
| **닫힘 (Closing)** | **팽창 후 침식** | 객체 내부의 작은 구멍(검은색 노이즈)을 메우고, 가까이 있는 객체들을 **하나로 합칩니다**. | cv2.morphologyEx(src, cv2.MORPH_CLOSE, kernel) |
| **경계 찾기 (Gradient)** | **(팽창) - (침식)** | 객체의 외곽선(Edge)만을 추출합니다. | cv2.morphologyEx(src, cv2.MORPH_GRADIENT, kernel) |

# 모폴로지 연산



# 경계선 및 윤곽선 검출

- 경계선 및 윤곽선 검출은 이미지에서 **객체와 배경을 분리**하고 객체의 외형적 특징(모양, 크기) 을 파악하는 데 사용되는 기초적인 컴퓨터 비전 기술입니다.
- **경계선 검출 (Edge Detection)**
  - 경계선(Edge)은 이미지의 **밝기 값(픽셀 값)이 급격하게 변하는 지점**을 의미하며, 이는 엣지 검출 필터(커널)를 사용하여 찾습니다.
  - **Canny 엣지 검출기 (Canny Edge Detector)**
  - Canny는 가장 널리 사용되고 효과적인 엣지 검출 알고리즘입니다. 노이즈에 강하고 얇고 정확한 엣지를 검출합니다.  흑백이미지에서 더 잘됨

# 경계선 및 윤곽선 검출

- **Canny 엣지 검출기 (Canny Edge Detector)**

| 단계 | 설명 |
|---|---|
| **1단계** | **노이즈 제거**: 가우시안 필터로 이미지의 노이즈를 제거합니다. |
| **2단계** | **경도 계산**: Sobel 필터 등을 사용하여 이미지의 밝기 변화율(Gradient)과 방향을 계산합니다. |
| **3단계** | **비최대 억제 (Non-Maximum Suppression)**: 엣지의 폭을 1픽셀로 줄여 얇은 엣지를 만듭니다. |
| **4단계** | **히스테리시스 임계 처리 (Hysteresis Thresholding)**: 두 개의 임계값(상위/하위)을 사용하여 최종 엣지를 연결하고 결정합니다. |

| 함수 | `cv2.Canny(image, threshold1, threshold2)` |
|---|---|
| `threshold1, threshold2` | 하위 임계값과 상위 임계값. 엣지 검출의 민감도를 조절합니다. |

# 경계선 및 윤곽선 검출

- **윤곽선 검출 (Contour Detection)**
- 윤곽선(Contour)은 **동일한 색상이나 밝기**를 가진 영역의 경계를 연결한 곡선입니다. 엣지 검출 결과(이진 이미지)를 입력으로 받아 객체의 **닫힌 경계선**을 찾아냅니다.
- **용도:** 이미지 내 객체의 위치, 크기, 모양, 방향 등을 파악하거나 객체 간의 위계를 분석하는 데 사용됩니다.

| 함수 | `cv2.findContours(image, mode, method)` |
|---|---|
| `mode` | 윤곽선을 찾는 방법 (예: `cv2.RETR_EXTERNAL` - 가장 바깥쪽 윤곽선만 검출) |
| `method` | 윤곽선을 저장하는 방법 (예: `cv2.CHAIN_APPROX_SIMPLE` - 중요한 점만 저장하여 메모리 절약) |

# 경계선 및 윤곽선 검출

- **출력:** 윤곽선 리스트와 계층 정보(Hierarchy)를 반환합니다.

| 함수 | `cv2.drawContours(image, contours, contourIdx, color, thickness)` |
|---|---|
| `contours` | `findContours`에서 반환된 윤곽선 리스트 |
| `contourIdx` | 그릴 윤곽선의 인덱스 (`-1`이면 모든 윤곽선) |

# 히스토그램 분석

- 히스토그램 분석은 이미지의 **픽셀 값 분포**를 시각화하고 정량적으로 분석하는 중요한 도구입니다. 이를 통해 이미지의 명암 대비, 밝기 수준, 그리고 전체적인 품질을 파악하고 조절할 수 있습니다.
- **히스토그램의 기본 개념**
- **정의:** 이미지의 각 밝기 레벨(0~255)에 해당하는 픽셀이 **몇 개** 존재하는지를 막대그래프로 나타낸 것입니다.
- **흑백 이미지:** 하나의 채널(밝기)에 대한 히스토그램이 생성됩니다.
- **컬러 이미지:** B, G, R 각 채널에 대해 별도의 히스토그램이 생성됩니다.

# 히스토그램 분석

- 히스토그램 해석 예시

| 분포 형태 | 이미지 특성 |
| --- | --- |
| **왼쪽(0)에 치우침** | 어두운 영역(그림자)이 많고, 전반적으로 **어두운 이미지** (명암 대비 낮음) |
| **오른쪽(255)에 치우침** | 밝은 영역(하이라이트)이 많고, 전반적으로 **밝은 이미지** (명암 대비 낮음) |
| **전체적으로 넓게 분포** | 명암 대비가 좋고, **다양한 밝기**를 포함하는 이미지 |

# 히스토그램 분석

- 히스토그램 계산 함수

| 함수 | 용도 | 구문 |
| --- | --- | --- |
| **cv2.calcHist()** | 히스토그램 계산 | cv2.calcHist(images, channels, mask, histSize, ranges) |

  - images: 계산할 원본 이미지 (NumPy 배열)
  - channels: 계산할 채널 인덱스 (예: [0]은 흑백 또는 B 채널)
  - mask: 마스크 이미지. 이미지의 특정 영역만 계산할 때 사용하며, 전체 이미지라면 None.
  - histSize: 막대(Bin)의 개수. 보통 256 (0~255의 모든 밝기 레벨)을 사용.
  - ranges: 픽셀 값의 범위 (보통 [0, 256] - 0부터 255까지).

# 히스토그램 분석

- **히스토그램 평활화 (Equalization)**
  - 히스토그램 평활화는 이미지의 대비(Contrast)를 개선하는 데 사용됩니다.
  - **목표:** 픽셀 값이 **특정 범위에 몰려 있는** 이미지를 **전체 범위(0~255)에 고르게** 퍼지도록 재분배하여 대비를 높입니다.
  - **효과:** 어둡거나 밝아 대비가 낮은 이미지를 더 보기 좋게 개선합니다.

| 함수 | cv2.equalizeHist() |
| --- | --- |
| **cv2.equalizeHist(src)** | 흑백 이미지의 히스토그램을 평활화하여 대비를 개선합니다. (컬러 이미지에는 바로 적용할 수 없으며, 흑백 변환 후 적용해야 함) |

# 특징점 검출 기초

- 특징점 검출은 이미지에서 **위치와 특징이 명확하게 정의되는 지점**을 찾는 것을 말합니다. 이 특징점들은 이미지의 크기, 회전, 밝기 변화 등에 강인하여, 객체 인식, 이미지 정합(Stitching), 3차원 복원 등의 고급 컴퓨터 비전 작업에 핵심적으로 사용됩니다.
- **특징점의 정의**
  - 특징점은 주변 픽셀과 구별되는 독특한 패턴을 갖는 이미지의 작은 영역입니다. 이상적인 특징점은 다음과 같은 속성을 가져야 합니다.
  - **반복성 (Repeatability):** 이미지의 크기나 시점(Viewpoint)이 바뀌어도 다시 검출되어야 합니다.
  - **지역성 (Locality):** 비교적 작은 영역에서 계산되어야 합니다.
  - **독특성 (Distinctiveness):** 다른 특징점과 쉽게 구별될 수 있어야 합니다.
  - 주요 특징점 유형은 다음과 같습니다.
  - **코너 (Corner):** 모든 방향으로 움직여도 픽셀 값이 크게 변하는 지점. (예: Harris 코너)
  - **블롭 (Blob):** 주변과 색상이나 밝기가 다른 영역. (예: SIFT, SURF, ORB)

# 특징점 검출 기초

- **특징점 검출 알고리즘**
  - **1) Harris 코너 검출기**
    - 가장 초기에 개발된 코너 검출 알고리즘 중 하나입니다.
    - **원리:** 작은 윈도우(커널)를 이미지 위에서 움직여 모든 방향으로의 **밝기 변화(경도)**를 측정합니다.
    - **특징:**
      - **평탄 영역:** 모든 방향으로 변화가 거의 없음.
      - **엣지 영역:** 엣지 방향과 수직인 방향으로만 변화가 큼.
      - **코너 영역:** 모든 방향으로 변화가 큼.
    - **함수:** cv2.cornerHarris(src, blockSize, ksize, k)

# 특징점 검출 기초

- **2) SIFT (Scale-Invariant Feature Transform)**
- 가장 유명하고 강력한 특징점 알고리즘 중 하나입니다.
- **특징:** 이미지의 크기 변화(Scale)와 회전(Rotation)에 모두 강인합니다. (즉, 객체를 멀리서 보거나 기울여서 봐도 같은 특징점을 찾습니다.)
- **원리:**
  - **스케일 공간 생성:** 여러 크기의 가우시안 블러를 적용한 이미지 피라미드를 만들어 크기 변화를 처리합니다.
  - **특징점 검출:** 각 스케일 공간에서 DoG(Difference of Gaussians)를 사용하여 극값(Maxima/Minima)을 찾습니다.
  - **기술자(Descriptor) 생성:** 특징점 주변 영역의 밝기 분포를 분석하여 128차원의 벡터로 변환합니다.
- **3) ORB (Oriented FAST and Rotated BRIEF)**
- SIFT와 SURF의 단점인 느린 속도와 특허 문제를 해결하기 위해 개발된 알고리즘입니다.
- **특징: 속도가 매우 빠르며** SIFT만큼은 아니지만 **회전과 밝기 변화**에 어느 정도 강인합니다. 상업적으로도 자유롭게 사용할 수 있습니다.
- **함수:** cv2.ORB_create()

# 비디오 처리 및 응용 프로젝트

# 비디오 스트림 처리(Video Stream Processing)

- OpenCV를 사용하는 주요 목적 중 하나는 실시간 비디오 또는 저장된 비디오 파일의 처리입니다. 비디오는 연속된 정지 이미지(프레임) 의 시퀀스이기 때문에, 기본적으로 이미지 처리에서 배운 모든 기술을 비디오의 각 프레임에 반복적으로 적용하여 처리합니다.

# 비디오 스트림 처리(Video Stream Processing)

- **비디오 입출력의 기본 구조**
- **1) 비디오 캡처 (VideoCapture)**
  - 카메라 또는 비디오 파일에서 프레임을 읽어오는 객체입니다.

| 함수/클래스 | 용도 | 설명 |
|---|---|---|
| `cv2.VideoCapture()` | 비디오 소스 열기 | **카메라 인덱스** (예: 0은 기본 카메라)나 **파일 경로**를 인자로 받습니다. |
| `.read()` | 프레임 읽기 | **`ret`** (성공 여부, True/False)와 **`frame`** (읽어온 이미지)을 반환합니다. |
| `.isOpened()` | 연결 확인 | 비디오 소스가 성공적으로 열렸는지 확인합니다. |
| `.release()` | 소스 해제 | 비디오 캡처 객체를 해제하여 카메라나 파일을 닫습니다. |

# 비디오 스트림 처리(Video Stream Processing)

- **비디오 출력 (VideoWriter)**
- 처리된 프레임들을 비디오 파일로 저장하는 객체입니다.

| 함수/클래스 | 용도 | 설명 |
| --- | --- | --- |
| **cv2.VideoWriter()** | 비디오 파일 생성 | **파일 이름**, **코덱**, **FPS**, **프레임 크기**를 인자로 받습니다. |
| **.write(frame)** | 프레임 쓰기 | 현재 프레임을 비디오 파일에 기록합니다. |

# 비디오.py

```python
# 1. 비디오 캡처 객체 생성 (0은 기본 웹캠을 의미)
cap = cv2.VideoCapture(0)

# 캡처가 성공적으로 열렸는지 확인
if not cap.isOpened():
    print("웹캠을 열 수 없습니다.")
    exit()

while True:
    # 2. 프레임 읽기
    # ret: 성공적으로 프레임을 읽었으면 True
    # frame: 읽어온 프레임 (컬러 이미지)
    ret, frame = cap.read()

    if not ret:
        print("프레임을 받을 수 없습니다. 종료합니다.")
        break

    # 3. 이미지 처리 적용: BGR -> Gray Scale 변환
    # 비디오의 모든 프레임에 이미지 처리 기술을 적용할 수 있습니다.
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 4. 화면에 결과 표시
    cv2.imshow('Original Video', frame)
    cv2.imshow('Gray Scale Video', gray_frame)

    # 5. 종료 조건 설정
    # 'q' 키를 누르면 루프를 종료합니다.
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# 6. 자원 해제
cap.release()
cv2.destroyAllWindows()
```

# 움직임 감지 및 추적

- 움직임 감지 및 추적은 컴퓨터 비전에서 가장 흔하게 사용되는 고급 기술 중 하나로, 비디오 스트림 내에서 **움직이는 객체를 식별**하고 **그 궤적을 따라가는** 것을 목표로 합니다.
- **움직임 감지 기초: 배경 차분 (Background Subtraction)**
  - 움직임을 감지하는 가장 기본적인 방법은 **배경 차분**입니다. 이는 현재 프레임과 배경 모델을 비교하여 차이가 나는 영역을 움직임으로 판단합니다.

# 배경 차분

- **단순 프레임 간 차분 (Simple Frame Differencing)**
  - **원리:** 이전 프레임과 현재 프레임 간의 **픽셀 값 차이**를 계산하여 변화가 큰 부분을 움직임으로 간주합니다.
  - **단점:** 카메라가 조금만 움직여도 전체 화면이 움직임으로 인식될 수 있고, 그림자 변화 등에도 민감하게 반응합니다.
- **배경 모델링을 통한 차분 (Background Modeling)**
  - **원리: 고정된 배경 모델**을 미리 구축하고, 현재 프레임과 이 배경 모델을 비교하여 움직이는 전경(Foreground) 객체만 분리합니다.
  - **OpenCV 함수:**
    - **cv2.createBackgroundSubtractorMOG2()**: 가우시안 혼합 모델(MOG2) 기반으로, 조명 변화나 배경의 작은 움직임에 더 강인한 배경 모델을 생성합니다.
    - **.apply(frame)**: 현재 프레임에서 배경을 제거하여 움직이는 객체만 포함된 **마스크(Mask)**를 반환합니다.

# 객체 추적 (Object Tracking)

- **객체 추적 (Object Tracking)**
  - 객체 추적은 움직임 감지 후, **특정 객체가 시간의 흐름에 따라 비디오 프레임 내에서 어디로 이동하는지** 정확히 예측하고 따라가는 과정입니다.
  - **용도:** 차량 번호판 추적, 사람의 움직임 분석, 스포츠 분석 등.
  - **OpenCV 기능:** OpenCV는 Tracker 클래스를 통해 **KCF, CSRT, GOTURN** 등 다양한 추적 알고리즘을 제공합니다.
    - **기본 과정:**
      - **초기화 (.init()):** 첫 프레임에서 추적할 객체의 **바운딩 박스**를 지정합니다.
      - **업데이트 (.update()):** 매 프레임마다 객체의 새로운 위치를 예측하고, 새로운 바운딩 박스를 반환합니다

# 객체 검출 및 인식 (AI 비전 입문)

- OpenCV를 활용한 최종 목표 중 하나는 이미지나 비디오에서 **특정 객체의 위치를 찾아내고 그 종류를 식별**하는 것입니다. 이 분야는 딥러닝 기술과 결합하여 발전했지만, OpenCV는 그 기반이 되는 전통적인 알고리즘과 **미리 학습된 모델을 활용할 수 있는 API**를 제공합니다.

# 객체 검출 및 인식 (AI 비전 입문)

- 객체 검출/인식의 차이

| 유형 | 목표 | 예시 |
| --- | --- | --- |
| **검출 (Detection)** | **객체의 위치**를 바운딩 박스로 표시하고 객체의 클래스(종류)를 식별합니다. | 이미지에서 '사람'은 어디에 있는지, '차량'은 어디에 있는지 찾음. |
| **인식 (Recognition)** | 객체가 무엇인지 식별하는 것에 초점을 맞춥니다. 검출 후, 그 객체가 **누구인지/무엇인지** 구체적으로 식별합니다. | 얼굴 검출 후, 그 얼굴이 '홍길동'임을 식별함 (예: 얼굴 인식). |

# 객체 검출 및 인식 (AI 비전 입문)

- 전통적인 검출 기법 (하르 캐스케이드)
  - OpenCV는 얼굴, 눈, 미소 등 미리 정의된 특정 특징을 검출하기 위해 **하르 캐스케이드(Haar Cascade)** 분류기를 제공합니다. 이는 딥러닝 이전 시대에 널리 사용되던 검출 방식입니다.
  - **원리:**
    - **Haar 특징:** 이미지의 특정 사각형 영역 간의 밝기 차이를 계산하여 객체의 특징(예: 눈 영역의 어두움, 코 영역의 밝음)을 나타냅니다.
    - **캐스케이드(Cascade):** 수많은 약한 분류기들을 계단식으로 연결하여, 배경 영역은 빠르게 버리고 객체가 있을 가능성이 높은 영역만 다음 단계로 넘겨 최종적으로 객체를 검출합니다.
  - **OpenCV 함수:**
    - **cv2.CascadeClassifier()**: 학습된 Haar 캐스케이드 XML 파일을 로드하여 분류기 객체를 생성합니다.
    - **.detectMultiScale()**: 이미지에서 객체를 검출하고, 검출된 객체의 바운딩 박스 리스트를 반환합니다.

# 객체 검출 및 인식 (AI 비전 입문)

- **딥러닝 기반 검출 및 인식 (DNN Module)**
  - 최근에는 YOLO, SSD, Faster R-CNN과 같은 **딥러닝(Deep Learning)** 모델이 검출 정확도와 속도 면에서 압도적인 성능을 보입니다. OpenCV는 이러한 모델들을 활용할 수 있는 **DNN(Deep Neural Network) 모듈**을 제공합니다.
  - **원리:** 미리 학습된 모델 파일(예: .weights, .cfg, .pb 파일)을 로드하여 이미지나 비디오 프레임에 적용합니다.
  - **OpenCV 함수:**
    - **cv2.dnn.readNet()**: TensorFlow, Caffe, Darknet 등 다양한 프레임워크로 학습된 신경망 모델을 로드합니다.
    - **cv2.dnn.blobFromImage()**: 이미지를 모델의 입력 요구사항에 맞는 형태로 전처리합니다.
    - **net.forward()**: 로드된 모델을 통해 순전파(Forward Propagation)를 수행하여 검출 결과를 얻습니다.

# 얼굴검출.py

```
# 2. 전처리: 흑백 변환 (Haar 캐스케이드는 흑백 이미지에서 더 빠름)
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 3. 얼굴 검출
# detectMultiScale(이미지, scaleFactor, minNeighbors)
# - scaleFactor: 이미지 크기 축소 비율 (1.1은 10%씩 축소)
# - minNeighbors: 최종적으로 객체로 인정하기 위한 최소 검출 횟수
faces = face_cascade.detectMultiScale(img_gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

# 4. 검출된 얼굴 영역에 사각형 그리기
for (x, y, w, h) in faces:
    # 원본 컬러 이미지에 초록색 사각형 그리기
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

# 사람 얼굴 및 신체 관련

| 파일 이름 | 검출 대상 | 용도 설명 |
|---|---|---|
| **haarcascade_frontalface_default.xml** | **정면 얼굴** | 가장 일반적이고 널리 사용되는 분류기입니다. 비교적 정면을 바라보는 얼굴을 검출합니다. |
| **haarcascade_frontalface_alt.xml** | **정면 얼굴 (대안)** | default보다 검출률은 낮을 수 있으나, 때로는 다른 각도의 얼굴이나 가려진 얼굴 검출에 더 효과적일 수 있습니다. |
| **haarcascade_frontalface_alt2.xml** | **정면 얼굴 (또 다른 대안)** | alt의 변형으로, 조금 더 다른 방식으로 학습된 얼굴 특징을 사용합니다. |
| **haarcascade_frontalface_alt_tree.xml** | **정면 얼굴 (Tree)** | 분류기로 **Decision Tree** 기반의 학습 방식을 사용한 버전입니다. |
| **haarcascade_profileface.xml** | **측면 얼굴** | 왼쪽 또는 오른쪽 측면을 향한 얼굴을 검출하는 데 특화되어 있습니다. |
| **haarcascade_eye.xml** | **눈** | 정면 얼굴이 검출된 영역 내에서 눈을 검출하는 데 주로 사용됩니다. |

# 사람 얼굴 및 신체 관련

| 파일 이름 | 검출 대상 | 용도 설명 |
|---|---|---|
| **haarcascade_lefteye_2splits.xml** | **왼쪽 눈** | 왼쪽 눈(사용자의 오른쪽 눈)을 검출합니다. |
| **haarcascade_righteye_2splits.xml** | **오른쪽 눈** | 오른쪽 눈(사용자의 왼쪽 눈)을 검출합니다. |
| **haarcascade_fullbody.xml** | **전신 (사람)** | 서 있는 사람의 전신을 검출합니다. |
| **haarcascade_upperbody.xml** | **상체** | 사람의 어깨 위 상체를 검출합니다. |
| **haarcascade_lowerbody.xml** | **하체** | 사람의 하체(허리 아래)를 검출합니다. |
| **haarcascade_smile.xml** | **미소/입** | 얼굴이 검출된 영역 내에서 미소를 짓는 입 모양을 검출합니다. |

# 기타 객체 및 동물 관련

| 파일 이름 | 검출 대상 | 용도 설명 |
|---|---|---|
| **haarcascade_car.xml** | **자동차** | 특정 각도의 자동차를 검출합니다. (정확도는 낮은 편) |
| **haarcascade_russian_plate_number.xml** | **러시아 차량 번호판** | 러시아 번호판의 특징에 맞춰 학습된 분류기입니다. |
| **haarcascade_frontalcatface.xml** | **고양이 얼굴 (정면)** | 정면을 바라보는 고양이 얼굴을 검출하는 데 사용됩니다. |
| **haarcascade_frontalcatface_extended.xml** | **고양이 얼굴 (확장)** | 기본 분류기보다 더 많은 고양이 얼굴 데이터를 사용하여 학습된 확장 버전입니다. |

- **성능:** Haar Cascade는 **속도가 빠르다**는 장점이 있지만, **정확도는 딥러닝 기반 모델(YOLO, Faster R-CNN)에 비해 낮습니다.** 특히 조명, 각도, 가림 등에 매우 민감합니다.
- **파일 위치:** 언급된 모든 파일이 사용자의 OpenCV 설치 환경에 포함되어 있지 않을 수 있습니다.
- 필요한 경우, **OpenCV 공식 GitHub 저장소**에서 다운로드하여 사용해야 합니다.
- https://github.com/opencv/opencv/blob/master/data/haarcascades

# OpenCV DNN 모듈

- **OpenCV DNN 모듈**은 **D**eep **N**eural **N**etwork의 약자로, OpenCV 라이브러리 내에서 **미리 학습된 딥러닝 모델**을 불러와서 실행할 수 있게 해주는 모듈입니다. 이 모듈의 주요 목표는 컴퓨터 비전 관련 딥러닝 기능을 복잡한 외부 프레임워크(TensorFlow, PyTorch 등)에 의존하지 않고 **OpenCV 환경 내에서 바로 사용할 수 있도록** 지원하는 것입니다.

# OpenCV DNN 특징

- 모델 호환성: 다양한 딥러닝 프레임워크에서 학습된 모델을 로드할 수 있습니다. 예를 들어, TensorFlow의 .pb 파일, Caffe의 .prototxt와 .caffemodel, PyTorch의 .pt 파일 등을 지원합니다.
- 추론(Inference) 실행: 모델을 학습시키는 기능은 제공하지 않지만, 이미지 분류, 객체 검출(YOLO, SSD), 시맨틱 분할 등 이미 학습이 완료된 모델을 사용하여 예측을 수행하는 데 특화되어 있습니다.
- 경량화 및 속도: 복잡한 딥러닝 프레임워크 전체를 설치하지 않아도 되므로, 비교적 경량화된 환경에서 딥러닝 추론을 실행할 수 있습니다.
- 하드웨어 가속: 기본적으로 CPU에서 실행되지만, Intel Movidius VPU나 특정 GPU 백엔드를 활용하여 추론 속도를 높이는 기능도 지원합니다.

# OpenCV DNN 사용 이유

- **Haar Cascade**는 빠르지만 정확도가 낮습니다. 최신 객체 검출 프로젝트(교통 검출 등)에서 **높은 정확도**를 얻으려면 YOLO나 SSD 같은 딥러닝 모델을 사용해야 합니다.
- OpenCV DNN 모듈을 사용하면:
- **복잡성 감소:** TensorFlow나 PyTorch 라이브러리 자체를 설치하고 관리하는 복잡한 과정 없이, **순수 OpenCV 코드**만으로 딥러닝 모델의 힘을 빌릴 수 있습니다.
- **통합 환경:** 비디오 스트림을 읽고, 이미지를 전처리하고, DNN 추론을 실행한 다음, 결과를 화면에 표시하는 **모든 과정을 OpenCV 내에서 일관되게 처리**할 수 있습니다.

# OpenCV DNN 모듈 실습 예제

- 이미지 분류
- **YOLOv8**
- **YOLOv8 ONNX 모델 파일:** yolov8n.onnx (YOLOv8 Nano 버전, 가장 가볍고 빠름)
- **클래스 레이블 파일:** coco.names (COCO 데이터셋 기준 80가지 객체 이름)
- **입력 이미지:** input_image.jpg (검출할 객체가 포함된 이미지)

**파일 다운로드**

- **yolov8n.onnx:** https://huggingface.co/SpotLab/YOLOv8Detection/blob/3005c6751fb19cdeb6b10c066185908faf66a097/yolov8n.onnx
- **coco.names:** https://raw.githubusercontent.com/pjreddie/darknet/master/data/coco.names
- coco.names 파일을 다운로드하여 coco.names로 저장하세요.
- **input_image.jpg:** 검출할 객체(사람, 자동차, 신호등 등)가 포함된 이미지를 준비하세요.

# YOLOv8n

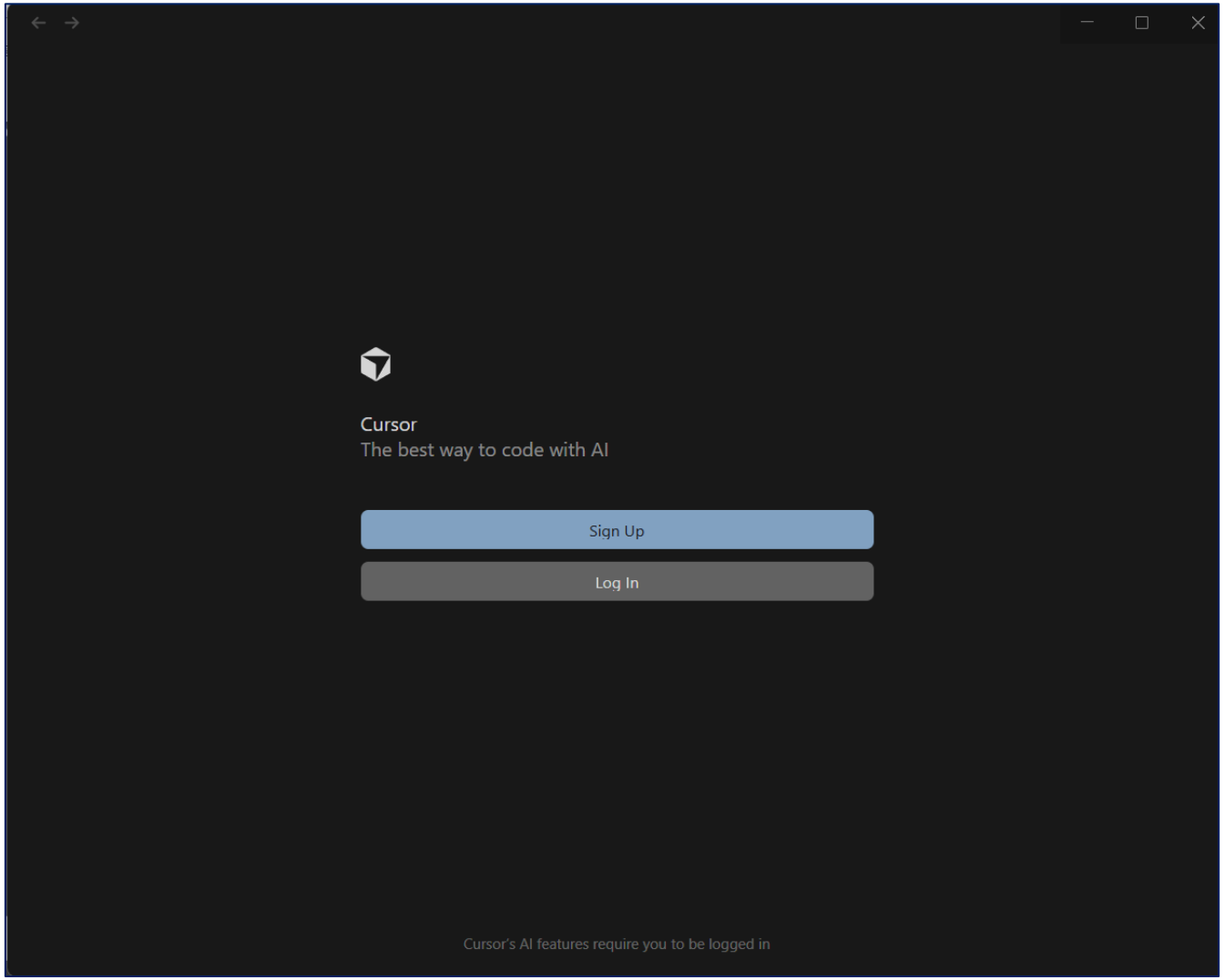
- **YOLOv8n**은 현재 가장 널리 사용되는 실시간 객체 검출 모델 시리즈인 **YOLO(You Only Look Once) 버전 8**의 **가장 작은 모델 크기**를 나타냅니다.

| 구성 요소 | 의미 | 설명 |
|---|---|---|
| **YOLO** | **Y**ou **O**nly **L**ook **O**nce | 검출과 분류를 단 하나의 신경망에서 동시에 처리하는 알고리즘 계열의 이름입니다 (1단계 검출기). |
| **v8** | **Version 8** | YOLO 알고리즘의 최신 주류 버전입니다. 이전 버전(v3, v4, v5, v7 등)에 비해 속도와 정확도가 크게 개선되었습니다. |
| **n** | **Nano** | YOLOv8 모델 계열 중 **가장 가벼운(경량) 모델**을 나타내는 접미사입니다. |

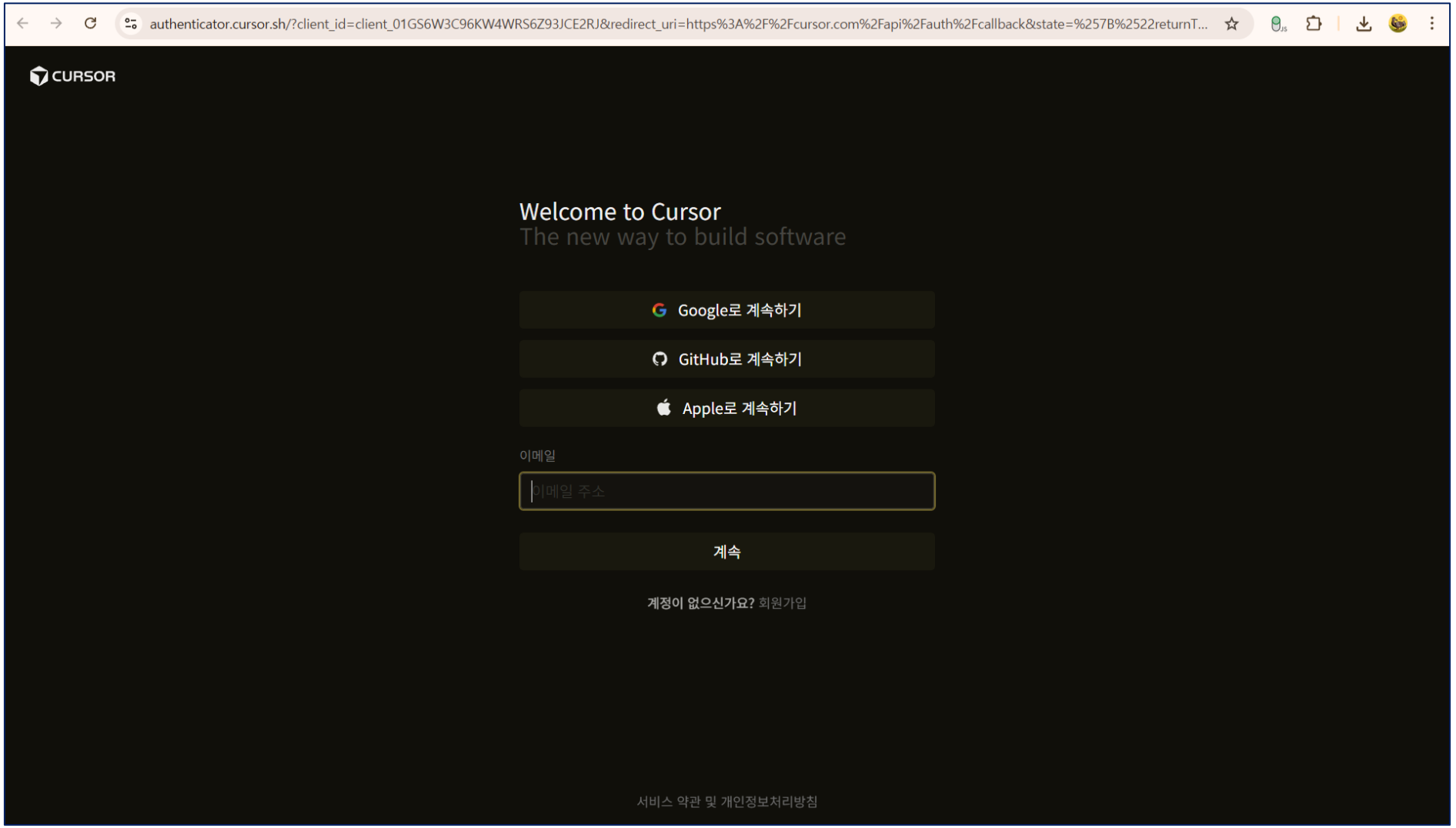
# 커서 AI 사용하기

- **설치 파일 다운로드 및 실행**
  - **공식 홈페이지 접속:** cursor.com에 접속합니다.
  - **Download 버튼 클릭:** 메인 화면에 있는 Download for Windows를 눌러 설치 파일을 받습니다.
  - **설치 실행:** 다운로드된 .exe 파일을 실행하면 별도의 복잡한 과정 없이 즉시 설치가 진행됩니다.

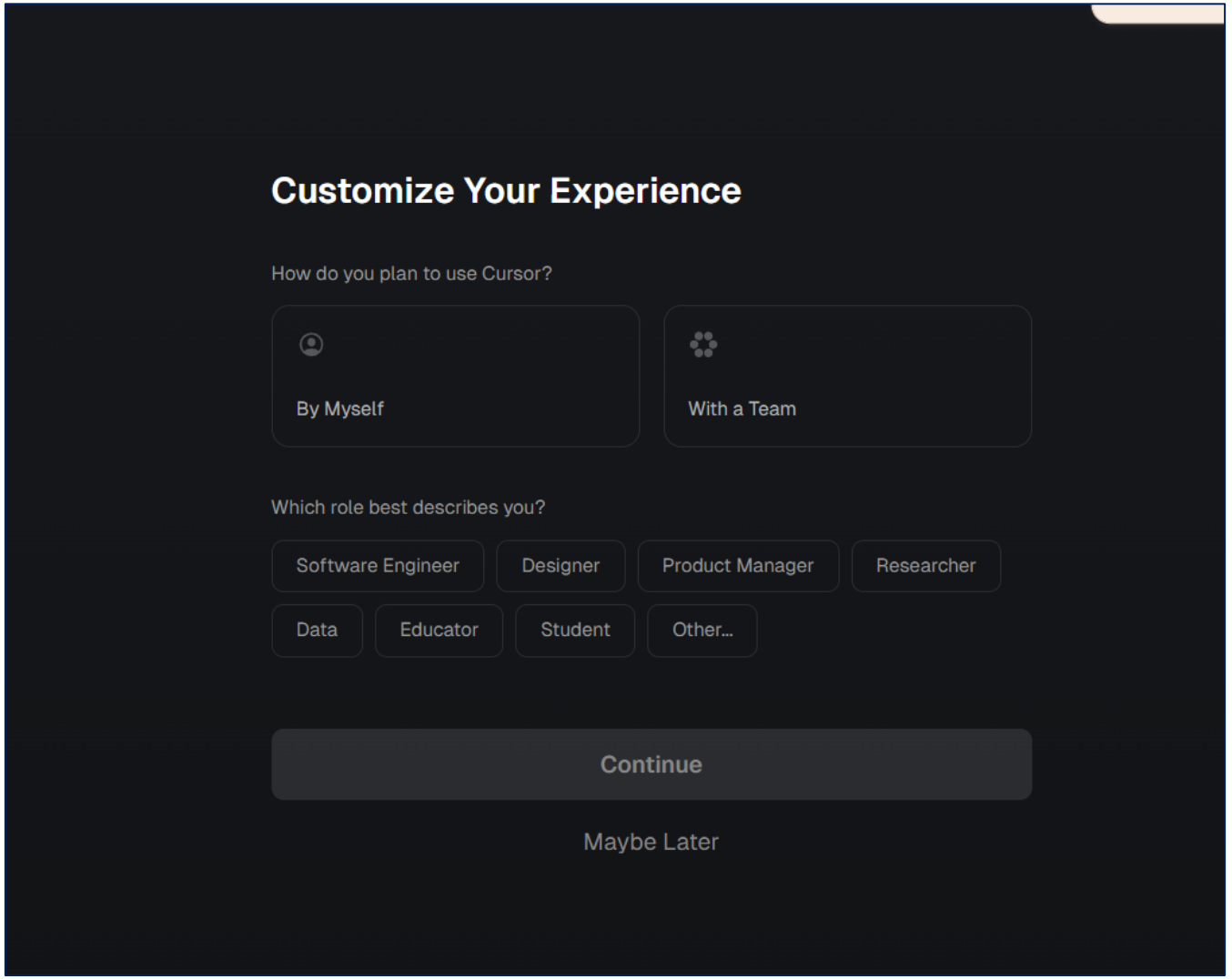
# 커서 AI 사용하기


Cursor
The best way to code with AI
Sign Up
Log In
Cursor's AI features require you to be logged in

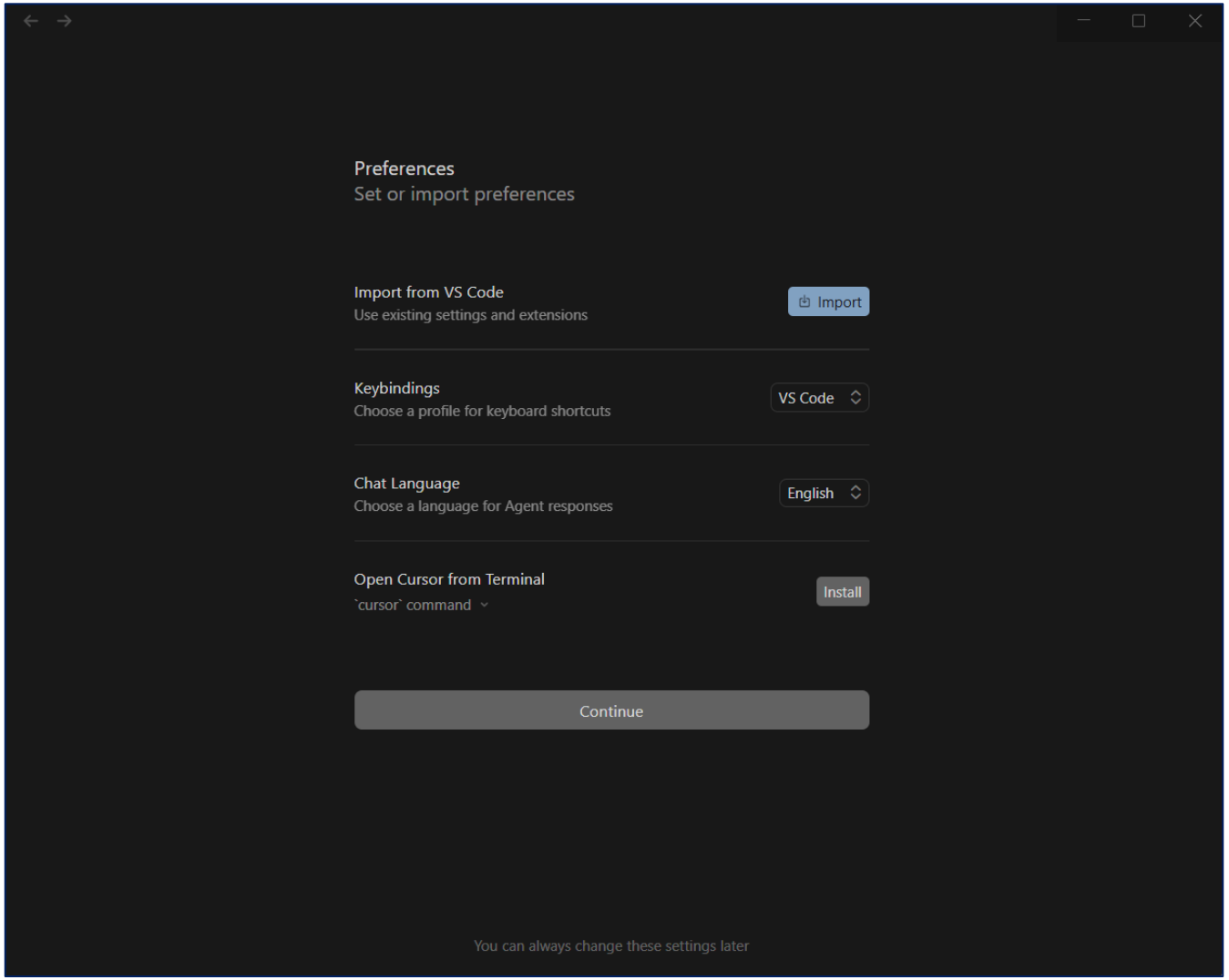
# 커서 AI 사용하기


authenticator.cursor.sh/?client_id=client_01GS6W3C96KW4WRS6Z93JCE2RJ&redirect_uri=https%3A%2F%2Fcursor.com%2Fapi%2Fauth%2Fcallback&state=%257B%2522returnT...
CURSOR
Welcome to Cursor
The new way to build software
Google로 계속하기
GitHub로 계속하기
Apple로 계속하기
이메일
이메일 주소
계속
계정이 없으신가요? 회원가입
서비스 약관 및 개인정보처리방침

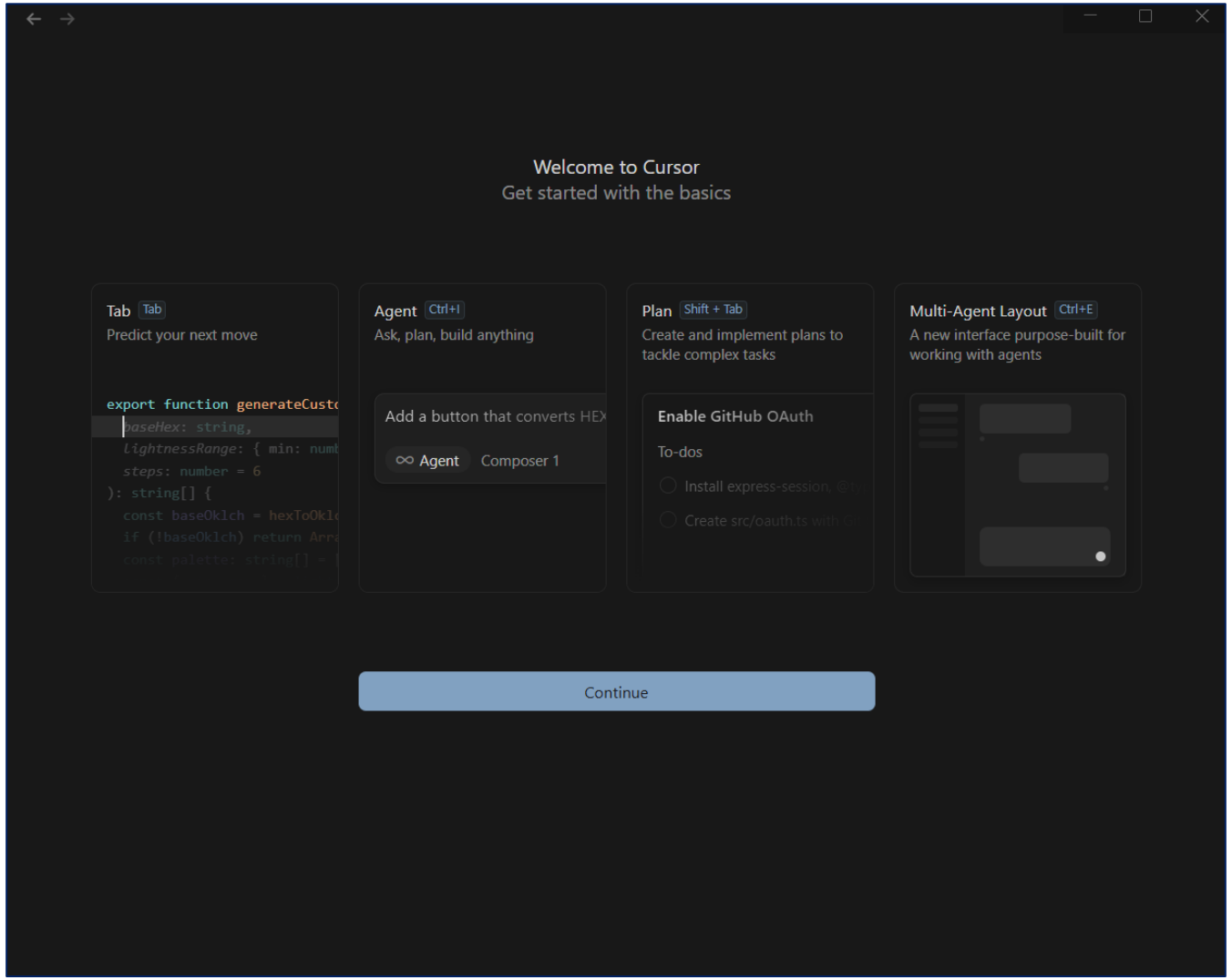
# 커서 AI 사용하기


Customize Your Experience
How do you plan to use Cursor?
By Myself
With a Team
Which role best describes you?
Software Engineer
Designer
Product Manager
Researcher
Data
Educator
Student
Other...
Continue
Maybe Later

# 커서 AI 사용하기


Preferences
Set or import preferences
Import from VS Code
Use existing settings and extensions
Import
Keybindings
Choose a profile for keyboard shortcuts
VS Code
Chat Language
Choose a language for Agent responses
English
Open Cursor from Terminal
`cursor` command
Install
Continue
You can always change these settings later

# 커서 AI 사용하기


Welcome to Cursor
Get started with the basics
Tab Tab
Predict your next move
Agent Ctrl+I
Ask, plan, build anything
Add a button that converts HEX
Agent
Composer 1
Plan Shift + Tab
Create and implement plans to tackle complex tasks
Enable GitHub OAuth
To-dos
Multi-Agent Layout Ctrl+E
A new interface purpose-built for working with agents
Continue

# 커서 AI 사용하기

- **초기 설정 (매우 중요!)**
  - 설치가 완료되고 처음 실행하면 설정 창이 뜹니다. 여기서 기존 VS Code의 설정을 가져와야 합니다.
  - **Keyboard Shortcuts:** Visual Studio Code를 선택합니다.
  - **VS Code Extensions:** **"Import Extensions"**를 꼭 선택하세요! (이걸 해야 지금껏 설치한 파이썬, 아나콘다 관련 도구들을 그대로 쓸 수 있습니다.)
  - **Language:** 한국어(Korean)로 설정하거나 기본값으로 두시면 됩니다.

# 커서 AI 사용하기

- **아나콘다 가상환경 연동 확인**
  - 커서는 VS Code 기반이라 가상환경 잡는 법이 똑같습니다.
  - 커서에서 파이썬 파일(.py)을 하나 엽니다.
  - 오른쪽 하단에 파이썬 버전이 적힌 부분을 클릭하거나, **Ctrl + Shift + P**를 누르고 **Python: Select Interpreter**를 입력합니다.
  - 아까 만드신 아나콘다 환경(예: vision_env 또는 base)을 선택합니다.
  - 이제 하단 터미널을 열면(Ctrl + `) 자동으로 해당 환경이 활성화됩니다.

# 커서 AI 사용하기

- **커서의 꽃, AI 기능 사용해보기**
  - 설치가 끝났다면 다음 두 가지만 먼저 테스트해보세요.
  - **에러 해결:** 아까처럼 OpenCV 에러가 나면, 터미널의 에러 메시지를 긁어서 **Ctrl + L**을 누르고 "이 에러 어떻게 고쳐?"라고 물어보세요.
  - **코드 생성:** Ctrl + K를 누르고 "OpenCV로 웹캠 띄우고 PyTorch 텐서로 변환하는 코드 짜줘"라고 입력해 보세요. AI가 실시간으로 코드를 작성해 줍니다.

# 커서 AI 사용하기

- **프로 플랜 (Pro)**
- 본격적으로 코딩을 많이 하는 분들이 사용합니다.
- **가격:** 월 $20 (약 2.7~2.8만 원)
- **제공량:**
  - **최고급 모델 (Claude 3.5 Sonnet 등):** 월 **500회** 빠른 응답 제공 (그 이후에도 무제한으로 쓸 수 있지만 속도가 느려질 수 있음)
  - **Composer:** 여러 파일을 동시에 수정하는 기능을 마음껏 사용 가능

- **무료 플랜 (Hobby)**
- 설치 직후 바로 적용되는 플랜입니다.
- **가격:** 0원
- **제공량:**
  - **Claude 3.5 Sonnet / GPT-4o:** 한 달에 약 **50회** 정도 고성능 모델 사용 가능 (이후에는 속도가 느려지거나 제한됨)
  - **작은 모델 (Cursor-small):** 무제한으로 사용 가능
  - **Cursor Tab:** 코드를 자동으로 완성해주는 기능도 2,000회 정도 제공

# 참고자료

- https://pyimagesearch.com/2018/07/19/opencv-tutorial-a-guide-to-learn-opencv/